

Lesson 10.2 — Demo Runbook (Авторизація: Policies, Gates, Form Requests)

Цей файл — єдина інструкція для онлайн-мітингу: **слайди -> цей runbook -> назад до слайдів**. Демо-проект — [workflow/practice-laravel/](#). Ми йдемо одразу після уроку 10.1, де додали [register/login/logout](#) через Sanctum. Сьогодні закриваємо «прірву»: доки немає полісу, будь-який авторизований юзер бачить чужі платежі.

0) Відповідність презентації

- **Слайд про різницю «authentication vs authorization»** → Блок А (демонструємо «дірку» на live-curl)
- **Слайд про PaymentPolicy і [viewAny/view/create/delete](#)** → Блок В (генеруємо й описуємо політику)
- **Слайд про [authorizeResource/Gate::authorize](#) у контролері** → Блок С (фіксуємо [PaymentController](#))
- **Слайд про [Gate](#) для адмін-дій** → Блок D (admin-only Gate [view-any-payment](#))
- **Фінальний чек-лист авторизації** → Блок Е

[AccountController](#), [PaymentService](#), репозиторії, [PaymentResource](#) **не переписуємо** — лише точково додаємо [authorize\(...\)](#) у потрібних місцях [PaymentController](#).

1) Pre-flight (до початку лекції)

1. [docker compose ps](#)
2. Потрібні маршрути 10.1 уже є:
 - [docker compose exec -T php php artisan route:list --path=api/v1/auth](#)
3. Прогон тестів:
 - [docker compose exec -T php php artisan test](#)
4. Telescope:
 - [http://localhost/telescope/requests](#)
5. Якщо [alice@example.com](#) / [bob@example.com](#) уже лежать у БД з минулого прогону — [register](#) поверне 422. Швидкий cleanup:

```
docker compose exec -T php php artisan tinker --
execute="\App\Models\User::whereIn('email',
['alice@example.com','bob@example.com'])->delete();"

```

6. Підготувати **двох** користувачів і одразу витягнути [token](#) у shell-змінну (на цьому й буде «дірка» в Блоці А):

```
ALICE_TOKEN=$(curl -s -X POST http://localhost/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"name":"Alice","email":"alice@example.com","password":"secret123"}' \
| jq -r '.data.token')

BOB_TOKEN=$(curl -s -X POST http://localhost/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"name":"Bob","email":"bob@example.com","password":"secret123"}' \
| jq -r '.data.token')

echo "ALICE_TOKEN=$ALICE_TOKEN"
echo "BOB_TOKEN=$BOB_TOKEN"
```

Далі по всьому уроку всі `curl` йдуть як `-H "Authorization: Bearer $ALICE_TOKEN"`.

Якщо `register` уже віддав `422` (юзер існує), беремо токен через `login`:

```
ALICE_TOKEN=$(curl -s -X POST http://localhost/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","password":"secret123"}' \
| jq -r '.data.token')
```

2) Блок А — демонстрація «дірки» без політики (перехід зі слайда `auth` vs `authz`)

2.1 Що говорити

- 10.1 закрит `auth` (хто ти такий). Не закрит `authz` (що тобі дозволено робити).
- Поки ми не додали `policy`, Alice через Bob's id бачить **чужі** дані. Це — типова **IDOR** (Insecure Direct Object Reference).

2.2 Підготовка демо-даних

В іншому терміналі — `tinker`:

```
docker compose exec -T php php artisan tinker
```

```
$alice = \App\Models\User::where('email', 'alice@example.com')->first();
$bob = \App\Models\User::where('email', 'bob@example.com')->first();

// Створюємо account для кожного (через існуючий ендпоінт або фабрику)
$accountA = \App\Models\Account::create(['balance' => 1000.00]);
$accountB = \App\Models\Account::create(['balance' => 1000.00]);
```

```
\App\Models\Payment::factory()->create([
    'user_id' => $alice->id, 'account_id' => $accountA->id,
    'amount' => 100, 'status' => 'processed', 'currency' => 'USD',
    'description' => 'Alice payment',
]);
\App\Models\Payment::factory()->create([
    'user_id' => $bob->id, 'account_id' => $accountB->id,
    'amount' => 200, 'status' => 'processed', 'currency' => 'USD',
    'description' => 'Bob secret payment',
]);

echo "alice_id={$alice->id} bob_id={$bob->id}";
exit
```

2.3 Live-демонстрація IDOR

Знаходимо id платежа Боба (для прикладу — припустимо `payment_id=2`).

```
# Alice заходить як вона сама — все ок
curl -i http://localhost/api/v1/payments/2 \
-H "Authorization: Bearer $ALICE_TOKEN"
```

Очікуємо: `200 OK` із даними Боба. Це і є **дірка**: будь-який авторизований юзер може дістати чужий платіж по id.

Що підкреслити:

- `auth:sanctum` лише сказав «ти Alice», але не сказав «тобі можна цей конкретний об'єкт».
- У продакшені це класичний привід для CVE/штрафу регулятора.

3) Блок В — генеруємо `PaymentPolicy` (перехід зі слайда про `*Policy`)

3.1 Що говорити

- Policy — стандартний Laravel-механізм, виділений під «що цьому юзеру можна з цією моделлю».
- Один клас на одну Eloquent-модель: `PaymentPolicy` для `Payment`. Методи: `viewAny`, `view`, `create`, `update`, `delete`.
- Регістрація автоматична, якщо клас лежить у `app/Policies/PaymentPolicy.php` і модель у `app/Models/Payment.php` (Laravel 12 «знаходить» policy через `ClassDiscovery`).

3.2 Генеруємо policy

```
docker compose exec -T php php artisan make:policy PaymentPolicy --
model=Payment
```

Відкриваю `app/Policies/PaymentPolicy.php` і вживу набираю наповнення:

```
<?php

declare(strict_types=1);

namespace App\Policies;

use App\Models\Payment;
use App\Models\User;

class PaymentPolicy
{
    public function viewAny(User $user): bool
    {
        return true;
    }

    public function view(User $user, Payment $payment): bool
    {
        return $payment->user_id === $user->id;
    }

    public function create(User $user): bool
    {
        return true;
    }

    public function update(User $user, Payment $payment): bool
    {
        return $payment->user_id === $user->id
            && $payment->status === 'pending';
    }

    public function delete(User $user, Payment $payment): bool
    {
        return $payment->user_id === $user->id
            && $payment->status === 'pending';
    }
}
```

Що підкреслити рядок за рядком:

1. `viewAny` повертає `true` — список повертаємо всім авторизованим, **але** в Блоці С ми обмежимо вибірку до власних платежів.
2. `view` — це і є фікс IDOR: відповідь `true` лише якщо `payment.user_id === user.id`.
3. `update/delete` додатково перевіряють стан: **процесований** платіж не редагуємо і не видаляємо. Це бізнес-правило, яке зручно тримати в одному місці з perm-перевіркою.

4) Блок С — підключаємо policy до `PaymentController` (перехід зі слайда про `authorize`)

4.1 Що говорити

- Policy без виклику нічого не робить. Підключити її можна двома шляхами:
 - `authorize('view', $payment)` у контролері — точково;
 - `authorizeResource(Payment::class, 'payment')` у конструкторі — на весь `apiResource`.
- Беремо обидва підходи, щоб студенти бачили різницю.

4.2 Правимо `PaymentController` (existing file — додаємо точково)

Відкриваю `app/Http/Controllers/Api/V1/PaymentController.php`. Поточний `index/store/show/destroy` залишається — додаємо виклики `authorize` і обмеження вибірки.

В `index` додаю фільтр + `authorize`:

```
public function index(Request $request): AnonymousResourceCollection
{
    $this->authorize('viewAny', Payment::class);

    $payments = Payment::query()
        ->where('user_id', $request->user()->id)
        ->latest()
        ->paginate(20);

    return PaymentResource::collection($payments);
}
```

В `show` — `authorize('view', ...)`. Поточна сигнатура контролера приймає `int $id`, але для policy краще route-model binding. Перепишу сигнатуру вживу:

```
public function show(Payment $payment): JsonResponse
{
    $this->authorize('view', $payment);

    return response()->json(['data' => new PaymentResource($payment)]);
}
```

В `destroy` — додаю `authorize('delete', $payment)`:

```
public function destroy(Payment $payment): JsonResponse
{
    $this->authorize('delete', $payment);

    $this->paymentService->deletePayment($payment);
}
```

```
return response()->json(null, 204);
}
```

В `store` — `authorize('create', Payment::class)` як приклад:

```
public function store(PaymentStoreRequest $request): JsonResponse
{
    $this->authorize('create', Payment::class);

    // ... існуючий код ...
}
```

Важлива деталь: `App\Http\Controllers\AntiPattern\Controller` (батьківський клас контролера в `practice-laravel`) **не** трейтить `AuthorizesRequests`. Live-fix:

```
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;

class PaymentController extends Controller
{
    use AuthorizesRequests;
    // ...
}
```

(Альтернатива — `\Illuminate\Support\Facades\Gate::authorize('view', $payment)`. Показую варіант із трейтом як стандартний.)

4.3 Перевіряємо, що `auth:sanctum` справді ввімкнений

`apiResource('payments', PaymentController::class)` у `routes/api.php` зараз — **не за `auth:sanctum`**. Треба обгорнути:

```
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('payments', PaymentController::class)-
>except(['update']);
    Route::apiResource('accounts', AccountController::class)-
>except(['update']);
});
```

Що підкреслити:

- Без `auth:sanctum` полісу впаде з помилкою (`$request->user()` буде `null`). 10.2 — момент, коли стейтові ендпоінти переїжджають за `auth`.
- Демо-ендпоінти `payments-fat`, `payments-cached`, `demo-fail` лишаємо публічними, бо вони лише для лекційної демонстрації.

4.4 Live-перевірка фіксу IDOR

```
# Alice -> власний платіж — 200
curl -i http://localhost/api/v1/payments/1 \
  -H "Authorization: Bearer $ALICE_TOKEN"

# Alice -> платіж Боба — 403 (policy спрацювала)
curl -i http://localhost/api/v1/payments/2 \
  -H "Authorization: Bearer $ALICE_TOKEN"

# Boba -> власний — 200
curl -i http://localhost/api/v1/payments/2 \
  -H "Authorization: Bearer $BOB_TOKEN"

# Без токена — 401
curl -i http://localhost/api/v1/payments/2
```

Очікуємо ланцюжок **200 / 403 / 200 / 401**. У Telescope у вкладці **Requests** — фільтр **Status 403** має містити одну запис; у вкладці **Gates** (якщо ввімкнено) — рядок із **payment.view → denied**.

5) Блок D — Gate для адмін-дій (перехід зі слайда **Gate::define**)

5.1 Що говорити

- Policy — для пар «модель + дія». Gate — для дій, що **не прив'язані** до конкретної моделі (наприклад, доступ до Telescope, до адмін-панелі, до глобальних агрегатів).
- Реєструємо Gate в **AppServiceProvider::boot()**. Перевіряємо так само через **\$this->authorize('view-all-payments')**.

5.2 Додаємо колонку **is_admin** (мінімально)

Швидка міграція (live-набір):

```
docker compose exec -T php php artisan make:migration add_is_admin_to_users
```

В свіжій міграції:

```
Schema::table('users', static function (Blueprint $table): void {
    $table->boolean('is_admin')->default(false);
});
```

```
docker compose exec -T php php artisan migrate
```

5.3 Реєструємо Gate в `AppServiceProvider::boot()`

```
use Illuminate\Support\Facades\Gate;
use App\Models\User;

Gate::define('view-all-payments', static function (User $user): bool {
    return (bool) $user->is_admin;
});
```

5.4 Адмін-ендпоінт «всі платежі»

Допишу метод у `PaymentController`:

```
public function adminIndex(Request $request): AnonymousResourceCollection
{
    $this->authorize('view-all-payments');

    $payments = Payment::query()->latest()->paginate(50);

    return PaymentResource::collection($payments);
}
```

Маршрут у `routes/api.php` (всередині `auth:sanctum`-групи, до `apiResource('payments', ...)`):

```
Route::middleware('auth:sanctum')->group(function () {
    Route::get('admin/payments', [PaymentController::class, 'adminIndex'])
        ->name('admin.payments.index');

    Route::apiResource('payments', PaymentController::class)-
>except(['update']);
    Route::apiResource('accounts', AccountController::class)-
>except(['update']);
});
```

Уважно з порядком: якщо `admin/payments` опиниться **після** `apiResource('payments', ...)`, Laravel зматчить його як `payments/{payment}` із `{payment}=admin` → `route-model binding` не знайде модель → 404 замість очікуваних 403/200.

5.5 Live-демо Gate

```
# Alice (не адмін) -> 403
curl -i http://localhost/api/v1/admin/payments \
-H "Authorization: Bearer $ALICE_TOKEN"
```



```
# Робимо Алісу адміном
docker compose exec -T php php artisan tinker --
execute="\App\Models\User::where('email','alice@example.com')-
>update(['is_admin' => true]);"

# Той самий запит — 200
curl -i http://localhost/api/v1/admin/payments \
-H "Authorization: Bearer $ALICE_TOKEN"
```

Що підкреслити:

- Gate без аргументу-моделі — це і є точка для глобальних правил.
- В реальному фінтеху замість `is_admin` буде роль із RBAC (Spatie Permission або власний `roles + role_user`). Принцип той самий.

6) Блок Е — фінальний чек-лист авторизації

- усі стейтові ендпоінти — за `auth:sanctum`
- кожна модель з персональними даними має `*Policy` з `view/update/delete`
- список (`index`) **обмежує вибірку** за `user_id`, навіть якщо `viewAny` повертає `true`
- адмін-дії — `Gate` + поле/роль на користувачі
- 401 — нема auth, 403 — auth є, але прав не вистачає; не плутати

7) Тести і логи наприкінці демо

1. Зелений прогон (сервіси, не зачеплені policy):

```
docker compose exec -T php php artisan test --filter PaymentService
```

2. Очікуване «червоне» після обгортання маршрутів у `auth:sanctum`:

```
docker compose exec -T php php artisan test --filter Payment
```

8 тестів упадуть з `401` (`CreatePaymentTest`, `CreatePaymentValidationTest`, `ShowPaymentNotFoundTest`) — вони анонімні, без `Sanctum::actingAs(...)`. Це **наочний приклад** того, що полісу-фікс ламає старі тести: їх треба оновити (Блок Е / домашка) додаванням:

```
protected function setUp(): void
{
    parent::setUp();
    \Laravel\Sanctum\Sanctum::actingAs(\App\Models\User::factory()
    >create());
}
```

3. `docker compose exec -T php sh -lc 'tail -n 100 storage/logs/laravel.log'`
4. У Telescope — фільтр за `Requests / Status 403` має показувати спробу IDOR з Блоку С.

8) Anti-fail (30 секунд до старту)

1. `docker compose exec -T php php artisan config:clear`
2. `docker compose exec -T php php artisan route:list --path=api/v1/payments`
3. Перевірити, що `AuthorizesRequests` уже у трейтах `PaymentController` (інакше `authorize()` упаде з `BadMethodCallException`).
4. Прогнати один auth-флоу:

```
curl -s -X POST http://localhost/api/v1/auth/login -H "Content-Type: application/json" \
  -d '{"email":"alice@example.com","password":"secret123"}'
```

9) Карта підготовленого коду під цей урок

Що вже є в `practice-laravel`

- Sanctum + `AuthController` із 10.1.
- `PaymentController` з `index/store/show/destroy` (без `authorize`).
- Маршрути `apiResource('payments', ...)` — публічні, без `auth:sanctum`.
- Модель `User` без поля `is_admin`.

Що набираю вживу під час мітингу

- `app/Policies/PaymentPolicy.php` (`viewAny/view/create/update/delete`).
- Виклики `$this->authorize(...)` у `PaymentController::index/show/store/destroy`.
- `use AuthorizesRequests` у `PaymentController`.
- Обгортання `apiResource('payments', ...)` і `apiResource('accounts', ...)` у `Route::middleware('auth:sanctum')`.
- Міграція `add_is_admin_to_users` + `Gate::define('view-all-payments')`.
- Адмін-ендпоінт `GET /api/v1/admin/payments`.

Що НЕ чіпаємо під час демо

- `PaymentService`, `PaymentRepository`, `PaymentResource`, `AccountController`, `ReportsController` — їх policy-перевірка не торкається.
- Демо-ендпоінти `payments-fat`, `payments-cached`, `demo-fail` — лишаються публічними.
- RBAC через Spatie Permission — обговорюємо словами, не встановлюємо в межах демо.